SOMS

Social Marketing Services

Build Specification: Database, Backend, Frontend and Build Order organized as independent systems

Stack: Express.js + JWT, PostgreSQL (Docker), React + Tailwind CSS, Cloudinary, Flask analytics service

Target market: Algeria | Version 1.0 | October 2026

Source: SOMS technical document (Arabic) + project brief. Sentiments module is OUT of scope.

Contents

Contents 2

1. Project Overview and Rules 5

1.1 Scope . . . 5

1.2 Golden rules (apply to every system) . . . 5

1.3 Decisions recorded in this spec (change them if you disagree) . . . 5

1.4 Open items you must resolve . . . 6

2. Architecture 7

2.1 Request flow for an analytics call . . . 7

2.2 Services and Docker . . . 7

2.3 Environment variables (.env.example; never commit the real .env) . . . 8

3. Technology Stack and Extra Tools 9

3.1 Your chosen stack . . . 9

3.2 Extra tools this project needs (recommended) . . . 9

3.3 Security middleware configuration . . . 9

4. Conventions Shared by All Systems 11

4.1 Response envelope . . . 11

4.2 Roles and permission matrix . . . 11

4.3 Tenancy middleware (the single place tenant scope is decided) . . . 11

4.4 Other conventions . . . 11

5. Project Structure and Build Order 13

5.1 Build order (one system at a time, test, then continue) . . . 13

5.2 Schema map (which system creates which table) . . . 14

System 0 - Foundation 15

Database . . . 15

Backend tasks and functions . . . 15

Endpoints . . . 15

Frontend tasks . . . 15

Flask tasks . . . 15

Acceptance . . . 15

System 1 - Authentication and Authorization 16

Database . . . 16

Backend: endpoints . . . 17

Backend: service functions . . . 17

Frontend pages and components . . . 17

Acceptance and tests . . . 18

System 2 - Company Registration and Administrative Requests 19

Database . . . 19

Backend: endpoints . . . 19

Backend: service functions . . . 20

Frontend pages . . . 20

Acceptance and tests . . . 20
System 3 - Users, Activity Logs, Settings and Admin Dashboard 21
Database . . . 21
Backend: endpoints . . . 22
Backend: service functions . . . 22
Frontend pages . . . 22
Acceptance and tests . . . 22
System 4 - Data Import (CSV), Public Posts, Wilayas and Demo Data 24
Database . . . 24
CSV templates (downloadable, one per type) . . . 25
Backend: endpoints . . . 26
Backend: service functions . . . 26
Frontend pages . . . 26
Acceptance and tests . . . 27
System 5 - Geo Analytics 28
Database . . . 28
Backend: endpoints . . . 28
Backend: service functions . . . 28
Frontend page: /analytics/geo . . . 28
Acceptance and tests . . . 29
System 6 - Growth and Market Expansion 30
Database . . . 30
Formulas . . . 30
Opportunity score . . . 30
Backend: endpoints . . . 30
Backend: service functions . . . 30
Frontend page: /analytics/growth . . . 30
Acceptance and tests . . . 31
System 7 - Scheduler Analytics and Internal Plan 32
Database . . . 32
Backend: endpoints . . . 32
Backend: service functions . . . 32
Frontend pages . . . 33
Acceptance and tests . . . 33
System 8 - Forecast (S-Curve Simulation) 34
Database . . . 34
Model . . . 34
Backend: endpoints . . . 34
Backend: service functions . . . 35
Frontend pages . . . 35
Acceptance and tests . . . 35
System 9 - Reports and Hub 36

Database . . . 36
Backend: endpoints . . . 36
Backend: service functions . . . 36
Frontend pages . . . 37
Acceptance and tests . . . 37
System 10 - Hardening, Testing and Delivery **38**
Security checklist . . . 38
Test strategy . . . 38
Delivery . . . 38
Final acceptance checklist (from the source document) . . . 38
Appendix A - Flask Analytics Service Contract **39**
Appendix B - Antigravity Prompts **40**
B.1 Master prompt (paste once / save as workspace rules) . . . 40
B.2 System prompt template (use for Systems 1 to 10) . . . 41
B.3 Ready-to-use prompt: System 0 and System 1 . . . 41
B.4 Review prompt (run after each system) . . . 41

1. Project Overview and Rules

SOMS is **one integrated web application** for the Algerian market that combines (a) an administrative management system (companies, users, join requests, permissions, logs) and (b) a marketing analytics platform (Hub, Forecast, Geo, Scheduler, Growth, Reports). It must be one product with one database and one login, not two projects.

1.1 Scope

Area	In scope (v1)	Out of scope
Administrative	Landing + registration request, login, admin dashboard, companies, requests, users/roles, settings, activity logs, AR/FR/EN	Billing / subscriptions (optional later)
Analytics	Hub, Forecast (S-Curve), Geo, Scheduler (analysis + internal plan), Growth, Reports (CSV)	Sentiments module (no page, table, route or function), automatic posting to platforms, PDF export (later)
Data	Public data entered manually, CSV import, synthetic demo data (clearly labelled)	Any private Meta/Instagram access, scraping behind login, bypassing platform terms

1.2 Golden rules (apply to every system)

- **Tenant isolation on the server.** Every query on company data filters by `company_id` taken from the verified JWT, never from the request body. Hiding a button is not security.
- **No invented numbers.** If data is missing or a denominator is zero, return `null` plus a reason; the UI shows "No data" / "Insufficient data". Never substitute 0 silently.
- **Every data row carries** `source_type`: `public_manual`, `csv_import`, `official_api` (future), `synthetic_demo`.
- **Demo data never mixes with real data.** Analytics endpoints take `dataMode=real|demo` (default `real`). Demo views show a permanent "DEMO DATA" banner and demo reports are watermarked.
- **Explainable results.** Every analytic result returns its inputs, formula/assumptions, data sources, sample size and last-updated time.
- **Estimates are labelled as estimates** (Forecast is a simulation, not a guarantee; Growth is based on the company data, not a full measure of the Algerian market).
- **No Meta access assumed.** The first release works with public data entered by hand or imported CSV files only.

1.3 Decisions recorded in this spec (change them if you disagree)

Topic	Decision	Reason
Database	PostgreSQL 16 in Docker from day one	Multi-tenant data; avoids SQLite to Postgres migration pain
IDs	UUID primary keys (`gen_random_uuid()`)	Not guessable, so changing an id in a URL reveals nothing
Company sign-up	Requires super-admin approval (creates an administrative request)	Matches "administrative requests" module; answers open question in the source document
Auth	JWT access token (15 min, kept in memory) + rotating refresh token (7 days, httpOnly cookie, stored hashed)	No token in localStorage; reuse detection possible
Flask role	Stateless compute service called only by Express (internal network, shared key). No DB access, no browser access	Keeps tenancy and permissions in ONE place (Express)
Super admin vs analytics	Super admin manages the platform; does not browse company analytics (only seeds demo data)	Privacy; matches role table in source document
Export	CSV (UTF-8 with BOM so Arabic opens in Excel) in v1; PDF later (print stylesheet meanwhile)	Source doc leaves PDF as optional
Thresholds (defaults)	Scheduler: min 5 posts per slot and 30 posts total; Growth: min 3 periods; Import: 5 MB / 50,000 rows	Stored in `system_settings`, editable by super admin

1.4 Open items you must resolve

#	Item	Needed before
1	GeoJSON file for Algerian wilaya boundaries and its source/licence. Algeria has 58 wilayas officially listed before 2019 and 69 after the 2019 reform; choose ONE list and use the same codes in the `wilayas` table, the CSV templates and the GeoJSON.	System 4 (seed) and System 5
2	Shape of your existing Flask algorithm code (function names, inputs, outputs) so it can be wrapped by the contracts in Appendix C.	System 6, 7, 8
3	Platforms in v1: Instagram first, Facebook second (recommended). TikTok optional.	System 4
4	Currency: assumed DZD for all money fields.	System 4
5	Email provider for password reset (optional; otherwise super admin resets passwords manually).	System 1

2. Architecture

```
Browser (React SPA + Tailwind, AR/FR/EN, RTL)
       |  HTTPS, JSON, access token in Authorization header, refresh token in httpOnly cookie
       v
Express API  --- helmet, cors(allowlist), rate-limit, JWT, zod validation, RBAC, tenancy scope
  |     |     \
  |     |      +--> Cloudinary (company logos, user avatars)
  |     +---------> Flask Analytics service (internal Docker network only, X-Internal-Key header)
  |                  forecast / scheduler / opportunity-score  (pure functions: JSON in, JSON out)
  v
PostgreSQL 16 (Docker volume)
```

2.1 Request flow for an analytics call

- React calls `GET /api/analytics/scheduler?...` with the access token.
- Express: `requireAuth` -> `requireRole` -> `resolveCompanyScope` (company id comes from the token) -> validates query with zod.
- Express loads only that company's rows from PostgreSQL (filtered by `dataMode` and `source_type`).
- Express sends the rows to Flask (`POST /compute/scheduler`) with the internal key; Flask returns the computed result.
- Express adds metadata (sources, last updated, warnings) and returns the standard response envelope.

2.2 Services and Docker

Service	Dev	Production
postgres:16	Docker container, volume `soms_pgdata`, port 5432 on localhost	Container, port NOT exposed publicly, daily `pg_dump` backup
pgadmin (optional)	Docker profile `dev`	Not deployed
api (Express)	Run locally with nodemon (or in compose)	Container behind reverse proxy (Caddy/Nginx, HTTPS)
analytics (Flask)	Run locally or in compose; no published port in compose	Container with gunicorn, internal network only
web (React)	Vite dev server	Static build served by Nginx/Caddy

```
# docker-compose.yml (development baseline)
services:
  postgres:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: soms
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: soms
    ports: ["5432:5432"]
    volumes: ["soms_pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U soms -d soms"]
      interval: 5s
      retries: 10
  pgadmin:
    image: dpage/pgadmin4
    profiles: ["dev"]
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASSWORD}
    ports: ["5050:80"]
volumes:
  soms_pgdata:
```

2.3 Environment variables (.env.example; never commit the real .env)

```
# --- API ---
NODE_ENV=development
PORT=4000
DATABASE_URL=postgresql://soms:CHANGE_ME@localhost:5432/soms
JWT_ACCESS_SECRET=CHANGE_ME_MIN_32_CHARS
JWT_ACCESS_TTL=15m
REFRESH_TTL_DAYS=7
COOKIE_SECURE=false
CORS_ORIGINS=http://localhost:5173
MAX_UPLOAD_MB=5
APP_TIMEZONE=Africa/Algiers
SUPERADMIN_EMAIL=admin@example.com
SUPERADMIN_PASSWORD=CHANGE_ME
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
ANALYTICS_URL=http://localhost:5000
INTERNAL_API_KEY=CHANGE_ME_LONG_RANDOM
# --- Web ---
VITE_API_URL=http://localhost:4000/api
# --- Flask ---
INTERNAL_API_KEY=CHANGE_ME_LONG_RANDOM
```

3. Technology Stack and Extra Tools

3.1 Your chosen stack

Layer	Tool	Notes
API	Express.js (JavaScript, ESM)	Layered: routes -> controllers -> services -> data access
Auth	JWT (`jsonwebtoken`) + refresh tokens	Passwords hashed with argon2id
Database	PostgreSQL 16 in Docker	Migrations are mandatory (no manual schema edits)
Frontend	React (Vite) + Tailwind CSS	RTL support via Tailwind logical utilities
Images	Cloudinary	Company logos and user avatars only
Analytics	Python + Flask	Your existing algorithm code wrapped as endpoints
Security	helmet, cors	Plus the extras below

3.2 Extra tools this project needs (recommended)

Need	Tool	Why
ORM + migrations	**Prisma** (or Knex/Drizzle if you prefer)	Typed queries, versioned migrations, easy seeding. SQL in this document is the reference model.
Input validation	**Zod**	Validates body/query/params on every route and the Flask responses
Password hashing	**argon2** (or bcrypt)	Requirement: hashed passwords only
Cookies	`cookie-parser`	Read the refresh-token cookie
Brute-force control	`express-rate-limit`	Strict limits on login, register, refresh, upload
HTTP hardening	`hpp`, `compression`	Parameter pollution guard, gzip
File upload	`multer` (memory storage) + `file-type`	Size/type limits; `file-type` checks magic bytes for images
CSV parsing	`csv-parse`	Streaming, supports `;` delimiter (Excel in Algeria/France exports with `;`) and BOM
Dates / timezone	`dayjs` + timezone plugin (or `date-fns-tz`)	Scheduler needs Africa/Algiers hour/weekday
Logging	`pino` + `pino-http`	Structured logs with request id; never log passwords/tokens
Calling Flask	`axios` with timeout	10 s timeout, retry once on network error
API docs	`swagger-ui-express` (optional)	Gives the examiner a live API reference
Email (optional)	`nodemailer`	Password reset / invitations
Server tests	Vitest or Jest + **Supertest**	Integration tests incl. tenant isolation
Frontend routing/data	React Router, **TanStack Query**, axios	Caching, loading and error states
Forms	**React Hook Form** + Zod resolver	Same schemas style as backend
i18n	**react-i18next**	AR / FR / EN; set `dir="rtl"` and `lang` on the html tag
Charts	**Chart.js** + react-chartjs-2	Line, bar; heatmap built as a custom grid component
Map	**Leaflet** + react-leaflet	Choropleth from GeoJSON
UI primitives	Headless UI or Radix, `lucide-react`, `react-hot-toast`, `react-day-picker`	Accessible modals, menus, icons, toasts, date range
Tailwind extras	`@tailwindcss/forms`, `clsx`, `tailwind-merge`	Consistent forms and class handling
Arabic font	`@fontsource/cairo` or Tajawal	Good Arabic + Latin rendering
Frontend tests	Vitest + React Testing Library, MSW	Mock API; optional Playwright for e2e
Flask side	Flask, gunicorn, pandas, numpy, scipy, pydantic, pytest, python-dotenv	scipy `curve_fit` only needed for optional forecast calibration
Code quality	ESLint, Prettier, Husky + lint-staged	Keeps Antigravity output consistent
CI / ops	GitHub Actions, Caddy or Nginx, `pg_dump` script	Run tests on push; HTTPS; backups
Dependency safety	`npm audit`, Dependabot	Part of the security checklist

3.3 Security middleware configuration

- **helmet**: default config for the API; if the API ever serves the React build, add a tuned Content-Security-Policy.

- **cors**: origin allowlist from `CORS_ORIGINS` (never `*` with credentials), `credentials: true`, methods GET/POST/PATCH/DELETE, headers `Content-Type, Authorization`.
- **Rate limits** (suggested): login 5 per 15 min per IP+email; register request 5 per hour per IP; refresh 30 per 15 min; upload 20 per hour per user; general 300 per 15 min.
- **Body limits**: JSON 1 MB; file uploads per `MAX_UPLOAD_MB`.
- **Cookies** for refresh token: `httpOnly`, `secure` in production, `sameSite=strict` (or `lax` if front and API are on different sites), `path=/api/auth`.
- **SQL injection**: ORM or parameterized queries only; never string-concatenate SQL.
- **XSS**: React escapes by default; never use `dangerouslySetInnerHTML` with user data (captions are user text).
- **Secrets**: only via env vars; `.env` is in `.gitignore`.

4. Conventions Shared by All Systems

4.1 Response envelope

```
Success:  { "success": true,  "data": { ... }, "meta": { "page": 1, "limit": 20, "total": 134 } }
Failure:  { "success": false, "error": { "code": "VALIDATION_ERROR", "message": "...", "details": [ ... ] } }
```

Error code	HTTP	Meaning
VALIDATION_ERROR	400	Body/query/params rejected by zod
UNAUTHENTICATED	401	Missing/expired/invalid token
FORBIDDEN	403	Role not allowed for this action
NOT_FOUND	404	Also used when a resource belongs to another company (do not leak existence)
CONFLICT	409	Duplicate (email, pending request, import duplicate)
INSUFFICIENT_DATA	422	Analytics cannot be computed; includes `missing` list and a user-facing reason
RATE_LIMITED	429	Too many requests
INTERNAL_ERROR	500	Unexpected; message is generic, details only in logs

4.2 Roles and permission matrix

Capability	super_admin	company_admin	analyst	viewer
Manage companies, join requests, global settings	Y	-	-	-
Manage all users	Y	-	-	-
Manage users of own company	-	Y	-	-
Edit own company profile and logo	-	Y	-	-
View activity logs	all	own company	-	-
Import CSV / enter public posts	-	Y	Y	-
Rollback an import	-	Y	-	-
View analytics (Hub, Geo, Growth, Scheduler)	-	Y	Y	Y
Create/save simulations and schedule plans	-	Y	Y	-
Create reports	-	Y	Y	-
View/export reports	-	Y	Y	Y
Seed / purge demo data	Y	-	-	-

Rule: `super_admin` has `company_id = NULL`; every other role must have a company. Enforce with a CHECK constraint and in code.

4.3 Tenancy middleware (the single place tenant scope is decided)

- `requireAuth`: verifies the access token, loads the user, rejects inactive users and users whose company is inactive.
- `requireRole(...roles)`: 403 if the role is not listed.
- `resolveCompanyScope` (= `scope_company_get` in the source document): sets `req.companyId = req.user.companyId`. Any `companyId` in body/query is ignored for company roles.
- Resource by id: always query with BOTH `id` and `company_id`. If no row, answer 404.
- Acceptance test: a user from company A requests a record id of company B and always gets 404.

4.4 Other conventions

- Timestamps are `timestamptz` stored in UTC; scheduler logic converts to `Africa/Algiers`.
- Dates for periods are plain `date`. Period validation (`range_date_get`): `start <= end`, both valid, max span 5 years.
- Pagination: `?page=1&limit=20` (max 100). Sorting: `?sort=created_at&order=desc` with a whitelist of sortable fields.
- Money: `numeric(14,2)` in DZD. Counts: `integer` with `CHECK (>= 0)`.

- Enums are implemented as PostgreSQL enums or text + CHECK; keep the lists below as the single source of truth.
- Naming: tables snake_case plural, columns snake_case, JSON keys camelCase in API responses.
- `divide_safe(n, d)` returns `null` when `d` is 0/null or `n` is null. Use it everywhere a ratio is computed.

Enum	Values
role	super_admin, company_admin, analyst, viewer
company_status	active, inactive
request_status	pending, approved, rejected
source_type	public_manual, csv_import, official_api, synthetic_demo
platform	instagram, facebook, tiktok, other
content_type	image, video, reel, carousel, story, text, other
import_type	social_posts, regional_metrics, sales_records
import_status	previewed, processing, completed, completed_with_errors, failed, cancelled
metric_type (regional)	impressions, clicks, conversions, spend, orders, revenue, followers, profit_margin
report_type	geo, growth, scheduler, forecast, combined

5. Project Structure and Build Order

```
soms/
  docker-compose.yml  .env.example  README.md  docs/ (this spec + ERD)
  api/                              # Express
    prisma/ (schema.prisma, migrations/, seed.js)
    src/
      app.js  server.js
      config/ (env.js, cors.js, helmet.js)
      middleware/ (requireAuth, requireRole, resolveCompanyScope, validate, rateLimiters, errorHandler)
      modules/
        auth/        (routes, controller, service, schema)
        company/  requests/  users/  logs/  settings/
        imports/  posts/  geo/  growth/  scheduler/  forecast/  reports/  hub/
      services/ (cloudinary.js, analyticsClient.js, activity.js)
      utils/ (apiResponse, AppError, divideSafe, dateRange, wilayaNormalize, logger)
    tests/
  analytics/                        # Flask (your algorithms)
    app.py  routes/  algorithms/  tests/  requirements.txt
  web/                              # React + Vite + Tailwind
    src/
      app/ (router, providers)  api/ (axios client, hooks)
      components/ (ui, charts, map, tables, feedback)
      features/ (auth, admin, company, imports, geo, growth, scheduler, forecast, reports, hub)
      i18n/ (ar.json, fr.json, en.json)
      layouts/ (PublicLayout, AuthLayout, DashboardLayout)
    public/geojson/
```

5.1 Build order (one system at a time, test, then continue)

#	System	You can demo when finished
0	Foundation (Docker, skeleton, shared middleware, layouts)	Health check works, empty app shell loads in AR/FR/EN
1	Authentication and Authorization	Log in/out, token refresh, role-protected routes
2	Company registration and administrative requests	Company requests access, admin approves, company admin can log in
3	Users, activity logs, settings, admin dashboard	Full admin back-office
4	Data import (CSV), public posts, wilaya reference, demo data	Import a CSV with accepted/rejected rows and error report
5	Geo analytics	Choropleth map of wilayas from imported data
6	Growth and market expansion	Growth KPIs and explained opportunity ranking
7	Scheduler analytics	Heatmap, best posting windows, internal plan
8	Forecast (S-Curve)	Simulate, save, compare scenarios
9	Reports and Hub	Generate/export reports, Hub landing page
10	Hardening, testing, delivery	Acceptance checklist green, README, production compose

5.2 Schema map (which system creates which table)

Table	System	Purpose
companies	1	Tenants
users	1	Accounts and roles
refresh_tokens	1	Rotating refresh tokens (hashed)
password_reset_tokens (optional)	1	Reset flow
administrative_requests	2	Company join requests and decisions
activity_logs	3	Audit trail
system_settings	3	Global thresholds and options
wilayas	4	Reference list of wilayas (code, names in 3 languages, aliases)
data_imports, data_import_errors	4	Import history and row-level errors
social_posts	4	Public post metrics (nullable metrics)
regional_metrics	4	Per-wilaya metrics per period
sales_records	4	Orders/revenue/acquisition cost
scheduled_items	7	Internal posting plan (no auto-publish)
campaign_simulations	8	Saved Forecast scenarios
reports	9	Generated report snapshots

Relations: `companies 1-N users`; `companies 1-N` every company-owned table; `data_imports 1-N` social_posts / regional_metrics / sales_records / data_import_errors; `wilayas 1-N` regional_metrics and sales_records; `users 1-N` refresh_tokens, imports, simulations, reports, activity_logs (as actor).

System 0 - Foundation

Goal: a running skeleton that every later system plugs into. No business features yet.

Database

No business tables. Initialize Prisma, create the first empty migration, add the seed script entry point (super admin is seeded in System 1).

Backend tasks and functions

Function	Behavior
`config/env.js` validateEnv()	Parses `process.env` with zod at startup; exits with a clear message if a variable is missing or weak (JWT secret < 32 chars).
`app.js` middleware order	`helmet` -> `cors` -> `hpp` -> `compression` -> `express.json({limit:"1mb"})` -> `cookieParser` -> `pino-http` -> general rate limiter -> routers under `/api` -> `notFound` -> `errorHandler`.
`utils/apiResponse` ok(res,data,meta) / fail(...)	Produces the standard envelope.
`utils/AppError`	Error class with `code`, `status`, `details`; thrown by services, converted by `errorHandler`.
`middleware/validate(schema)`	Validates `{body, query, params}`; replaces them with parsed values.
`middleware/errorHandler`	Maps AppError, zod errors and Prisma errors (unique violation -> CONFLICT) to the envelope; hides internals in production.
`middleware/rateLimiters`	Exports `general`, `login`, `register`, `refresh`, `upload`.
`utils/divideSafe`, `utils/dateRange`	Shared helpers from the source document (`divide_safe`, `range_date_get`) with unit tests.
`services/analyticsClient`	axios instance to Flask with base URL, `X-Internal-Key`, 10 s timeout, error mapping to INTERNAL_ERROR / service unavailable.

Endpoints

Method	Path	Access	Behavior / rules
GET	/api/health	Public	Returns API status and database connectivity (`SELECT 1`). Optionally Flask `/health`.

Frontend tasks

- Vite + React + Tailwind setup; Router with three layouts: `PublicLayout` (landing, register, login), `AuthLayout`, `DashboardLayout` (sidebar + topbar + language switcher + user menu).
- i18n with AR (default), FR, EN. Switching language sets `document.documentElement.dir` (`rtl` for AR) and `lang`. Use Tailwind logical classes (`ms-`, `me-`, `ps-`, `pe-`, `text-start`) so layouts flip correctly.
- axios client with base URL, `withCredentials: true`, response interceptor placeholders (completed in System 1).
- Shared UI kit: Button, Input, Select, Modal, Drawer, Table, Badge (incl. DataSourceBadge), EmptyState, Spinner, Toast, ConfirmDialog, Pagination.
- Pages: `/404`, `/403`, and a placeholder dashboard.

Flask tasks

- `app.py` with `/health`; before-request hook that rejects calls without a valid `X-Internal-Key` (except `/health`).
- Folder layout `routes/` + `algorithms/` (pure functions) + `tests/`; gunicorn command documented.

Acceptance

- `docker compose up -d postgres` then migration runs without errors.
- `GET /api/health` returns OK; stopping Postgres makes it report the database as down.
- Frontend loads; language switch flips the layout direction; unknown route shows 404 page.
- Flask rejects requests without the internal key.

System 1 - Authentication and Authorization

Goal: secure login with JWT, role-based access, and the tenancy middleware every later system depends on. This system also creates `companies` (needed by `users`); company management itself is System 2.

Database

companies

Column	Type	Constraints / notes
id	uuid PK	default gen_random_uuid()
name	varchar(150)	NOT NULL
registration_number	varchar(50)	UNIQUE, nullable (commercial register number)
industry	varchar(100)	nullable
email, phone	varchar	contact info, nullable
address	text	nullable
logo_url, logo_public_id	text	Cloudinary URL and public id, nullable
status	company_status	NOT NULL default 'active'
created_at, updated_at	timestamptz	default now()

users

Column	Type	Constraints / notes
id	uuid PK	
company_id	uuid FK -> companies	nullable; NULL only for super_admin
full_name	varchar(120)	NOT NULL
email	varchar(255)	NOT NULL; UNIQUE on lower(email)
password_hash	text	argon2id hash only
role	role enum	NOT NULL
is_active	boolean	default true
avatar_url, avatar_public_id	text	Cloudinary, nullable
preferred_language	varchar(2)	default 'ar' (ar/fr/en)
failed_login_count	int	default 0
locked_until	timestamptz	nullable; set after 5 failed attempts (15 min)
last_login_at	timestamptz	nullable
created_at, updated_at	timestamptz	

Constraints: `CHECK ((role = 'super_admin') = (company_id IS NULL))`. Indexes: `users(company_id)`, unique `lower(email)`.

refresh_tokens

Column	Type	Constraints / notes
id	uuid PK	
user_id	uuid FK -> users	ON DELETE CASCADE
token_hash	char(64)	SHA-256 of the random token; the raw token exists only in the cookie
family_id	uuid	groups a rotation chain; reuse of a revoked token revokes the whole family
expires_at, revoked_at	timestamptz	revoked_at nullable
replaced_by	uuid	nullable
user_agent, ip	text	nullable, for session list/audit
created_at	timestamptz	

password_reset_tokens (optional, only if email is used)

Column	Type	Constraints / notes
id, user_id, token_hash, expires_at (30 min), used_at, created_at		Single-use; store hash only

Backend: endpoints

Method	Path	Access	Behavior / rules
POST	/api/auth/login	Public, rate-limited	Body `{email, password}`. Generic error "Invalid credentials" for unknown email or wrong password. Blocks inactive user/company. Returns `{accessToken, user}` and sets the refresh cookie.
POST	/api/auth/refresh	Cookie	Validates and rotates the refresh token; returns a new access token. Reuse of a revoked token revokes the family and returns 401.
POST	/api/auth/logout	Authenticated	Revokes the current refresh token and clears the cookie.
GET	/api/auth/me	Authenticated	Current user + company summary + permissions list.
PATCH	/api/auth/me	Authenticated	Update own `full_name`, `preferred_language`; avatar via `POST /api/auth/me/avatar` (multipart, Cloudinary).
POST	/api/auth/change-password	Authenticated	Needs current password; new password policy (min 10 chars, not equal to email); revokes other sessions.
POST	/api/auth/forgot-password, /reset-password	Public (optional)	Always answers 200 on forgot; single-use token.

Backend: service functions

Function	Behavior
authService.login(email, password, ctx)	Lookup by lower(email); check lock; verify argon2; reset or increment failed counter; check `is_active` and company status; issue tokens; log `auth.login` / `auth.login_failed`.
hashPassword / verifyPassword	argon2id wrappers; the only place that touches password hashing.
issueTokens(user, ctx)	Access JWT (HS256, 15 min, claims `sub`, `role`, `companyId`) + random 256-bit refresh token (hash stored).
rotateRefreshToken(rawToken, ctx)	Find by hash, verify not revoked/expired, revoke, create next in same family; if already revoked -> revokeFamily.
revokeFamily(familyId), logout(rawToken)	Sets `revoked_at`.
getMe(userId), updateMe(userId, data)	Profile read/update.
changePassword(userId, current, next)	Verifies current, validates policy, updates hash, revokes sessions.
middleware requireAuth / requireRole / resolveCompanyScope	See section 4.3. `permissions.js` holds the permission matrix and `can(user, capability)`.
seed: ensureSuperAdmin()	Creates the super admin from `SUPERADMIN_EMAIL` / `SUPERADMIN_PASSWORD` if absent (hashes the password).

Frontend pages and components

Route	Page	Details
/login	Login	Email + password form, error messages, language switcher, link to company registration. After login: super_admin -> `/admin/dashboard`, others -> `/hub`.
/forgot-password, /reset-password/:token	Reset (optional)	Only if email is enabled.
/profile	My account	Edit name, language, avatar (preview + upload), change password.
/403, /404	Error pages	

- `AuthProvider`: keeps the access token in memory; on app load calls `/auth/refresh` silently to restore the session.
- axios interceptor: on 401, call refresh once, retry the original request, otherwise redirect to `/login`. Queue concurrent requests while refreshing.
- `ProtectedRoute` (requires login) and `RoleGuard roles={[...]}` (requires role). Sidebar items are filtered by role (UX only; the server enforces).

Acceptance and tests

- Correct login works; wrong password and unknown email give the same message; 5 failures lock the account.
- Refresh rotation works; reusing an old refresh token revokes the session family.
- Inactive user or user of an inactive company cannot log in or use an existing access token.
- Role guard: a `viewer` calling an `analyst` endpoint gets 403.
- Cross-company id access returns 404 (add the reusable test helper now; reuse in all later systems).
- Password never appears in responses or logs.

System 2 - Company Registration and Administrative Requests

Goal: a company asks to join, the super admin approves or rejects, and approval creates the company and its first company admin. Also companies CRUD for the super admin and the company profile page.

Database

administrative_requests

Column	Type	Constraints / notes
id	uuid PK	
company_name	varchar(150)	NOT NULL
registration_number	varchar(50)	nullable
industry	varchar(100)	nullable
contact_phone	varchar(30)	nullable
applicant_full_name	varchar(120)	NOT NULL
applicant_email	varchar(255)	NOT NULL
password_hash	text	argon2id hash of the password chosen at registration; copied to `users` on approval; cleared after decision
status	request_status	default 'pending'
reviewed_by	uuid FK -> users	nullable
reviewed_at	timestamptz	nullable
decision_note	text	nullable; shown to applicant if you add email later
created_company_id	uuid FK -> companies	nullable; set on approval
created_at	timestamptz	

Indexes: `(status, created_at desc)`; partial unique index on `lower(applicant_email)` where `status = 'pending'` (one pending request per email).

Backend: endpoints

Method	Path	Access	Behavior / rules
POST	/api/public/company-requests	Public, rate-limited	Validates input; rejects if email already exists in `users` or has a pending request (CONFLICT); stores the hashed password; logs `request.created`. Returns a generic success message.
GET	/api/admin/requests	super_admin	Filters `status`, `search`, pagination.
GET	/api/admin/requests/:id	super_admin	Detail.
PATCH	/api/admin/requests/:id	super_admin	Body `{decision: approved|rejected, note}`. Only pending requests. Approve runs ONE transaction: create company, create `company_admin` user from the request, update request, write log. Reject stores note and clears the hash.
GET	/api/admin/companies	super_admin	Search by name/registration number, filter by status, pagination, counts of users.
POST	/api/admin/companies	super_admin	Create company directly (optionally with first admin user and temporary password flagged for change).
GET	/api/admin/companies/:id	super_admin	Details + user count + recent activity.
PATCH	/api/admin/companies/:id	super_admin	Edit data.
PATCH	/api/admin/companies/:id/status	super_admin	Activate/deactivate. Deactivation blocks login and tokens of all its users. No hard delete in v1.
GET / PATCH	/api/company	company_admin (GET: any company role)	Own company profile.
POST	/api/company/logo	company_admin	Multipart image (jpeg/png/webp, max 2 MB) uploaded to Cloudinary folder `soms/companies/{companyId}`; old image deleted.

Backend: service functions

Function	Behavior
requestService.submit(data)	Validation, duplicate checks, hash password, insert.
requestService.decide(id, reviewerId, decision, note)	Transactional approval/rejection as described above; idempotent guard (status must be pending).
companyService.list / get / create / update / setStatus	CRUD with activity logging (`company.created`, `company.updated`, `company.status_changed`).
cloudinaryService.uploadImage(buffer, folder)	Uses `upload_stream`; applies allowed formats; returns `{url, publicId}`.
cloudinaryService.deleteImage(publicId)	Called when an image is replaced or removed.
upload middleware	`multer` memory storage, size limit, mime allowlist, magic-byte check with `file-type`.

Frontend pages

Route	Page	Details
/	Landing / welcome	What SOMS is, module overview, calls to action (Login, Request access). Language switcher.
/register	Company request form	Company name, registration number, industry, applicant name, email, phone, password + confirmation; client and server validation.
/register/success	Pending message	Explains that an admin must approve the request.
/admin/requests	Administrative requests	Tabs Pending / Approved / Rejected, search, table, detail drawer, Approve and Reject modals (note required for reject).
/admin/companies	Companies	Table with search/status filter, add and edit modal, activate/deactivate toggle with confirm dialog.
/admin/companies/:id	Company details	Info, status, users count, recent activity.
/company/settings	Company profile	Edit company info, upload/replace logo with preview.

Acceptance and tests

- Request -> approval -> the applicant can log in as `company_admin` with the password chosen at registration.
- Duplicate pending request for the same email is rejected; approving twice is impossible.
- Deactivated company: its users get 401/403 immediately.
- Logo upload rejects non-images and files over the limit; old Cloudinary image is deleted on replace.
- Company admin can edit only own company (`/api/company` ignores any id).

System 3 - Users, Activity Logs, Settings and Admin Dashboard

Goal: complete the administrative back-office: user and role management, audit trail, global settings and the admin dashboard.

Database

activity_logs

Column	Type	Constraints / notes
id	uuid PK (or bigserial)	
actor_user_id	uuid FK -> users	nullable (e.g. failed login of unknown email)
company_id	uuid FK -> companies	nullable; used to filter company logs
action	varchar(60)	e.g. 'user.created'
entity_type, entity_id	varchar / uuid	nullable
metadata	jsonb	safe details only; NEVER passwords, tokens, secrets
ip	varchar(45)	nullable
created_at	timestamptz	index `(company_id, created_at desc)` and `(action, created_at desc)`

Action names: `auth.login`, `auth.login_failed`, `auth.logout`, `company.created|updated|status_changed`, `request.created|approved|rejected`, `user.created|updated|role_changed|status_changed|password_reset`, `import.created|completed|failed|rolled_back`, `simulation.created`, `report.created`, `settings.updated`.

system_settings

Column	Type	Constraints / notes
key	varchar(80) PK	
value	jsonb	
updated_by	uuid FK -> users	nullable
updated_at	timestamptz	

Seeded keys: `default_language`, `allow_registration`, `import_max_file_mb` (5), `import_max_rows` (50000), `scheduler_min_posts_per_slot` (5), `scheduler_min_total_posts` (30), `growth_min_periods` (3), `opportunity_weights` ({demand_growth:30, conversion_rate:25, leads_volume:20, cac:15, margin:10}).

Backend: endpoints

Method	Path	Access	Behavior / rules
GET	/api/admin/users	super_admin	Filters `companyId`, `role`, `isActive`, `search`; pagination.
POST	/api/admin/users	super_admin	Create user for a company (or another super_admin). Temporary password shown once.
PATCH	/api/admin/users/:id	super_admin	Edit name, role, company.
PATCH	/api/admin/users/:id/status	super_admin	Activate/deactivate; revokes the user's refresh tokens when deactivated.
POST	/api/admin/users/:id/reset-password	super_admin	Generates a temporary password (shown once) and revokes sessions.
GET	/api/company/users	company_admin	Users of own company only.
POST	/api/company/users	company_admin	Create `company_admin | analyst | viewer` in own company.
PATCH	/api/company/users/:id	company_admin	Change name/role (never to super_admin).
PATCH	/api/company/users/:id/status	company_admin	Activate/deactivate. Cannot deactivate yourself or the last active company_admin.
GET	/api/admin/activity-logs	super_admin	Filters actor, company, action, date range; pagination.
GET	/api/company/activity-logs	company_admin	Own company only.
GET	/api/admin/settings	super_admin	All settings.
PATCH	/api/admin/settings/:key	super_admin	Validated per key (zod schema per setting); logs `settings.updated`.
GET	/api/settings/public	Public	Default language and registration availability.
GET	/api/admin/dashboard	super_admin	KPIs: active companies, pending requests, total/active users, active simulations (wired in System 8; returns 0 before), last 10 events.

Backend: service functions

Function	Behavior
activityService.record({actor, companyId, action, entityType, entityId, metadata, ip})	= `activity_record`. Never throws into the request flow (log and continue). Strips keys named password/token/secret.
userService.create / update / setStatus / resetPassword	Enforces the guards: no self-deactivation, no removing the last company admin, company admins cannot assign super_admin, role/company CHECK respected.
settingsService.get(key) / set(key, value, userId)	In-memory cache with invalidation on write; typed defaults.
statsService.getAdminDashboard()	Aggregate counts using one or two queries; no per-row loops.
logService.list(filters, scope)	Scope is `all` (super admin) or a company id.

Frontend pages

Route	Page	Details
/admin/dashboard	Admin dashboard	KPI cards, pending requests widget (links to requests), recent activity list, optional small charts (companies per month).
/admin/users	Users	Table with filters, create/edit modal, role selector, activate/deactivate, reset password (one-time display dialog).
/admin/logs	Activity logs	Filterable table with date range and action filter, expandable metadata.
/admin/settings	System settings	Sections: General (default language), Thresholds (scheduler/growth/import), Opportunity weights (sum must equal 100), Security info.
/company/users	Company users	Same table scoped to the company with allowed roles only.
/company/activity	Company activity	Company-scoped log table.

Acceptance and tests

- A company admin cannot see or edit users of another company (404).
- Cannot remove the last company admin or deactivate yourself.
- Deactivated user is logged out on next refresh.

- Logs are written for every sensitive action and never contain secrets.
- Settings validation rejects weights that do not sum to 100.

System 4 - Data Import (CSV), Public Posts, Wilayas and Demo Data

Goal: the only way analytics get data. Safe CSV import with preview, validation, accepted/rejected row counts and an error report; manual entry of public posts; a wilaya reference table; clearly labelled synthetic demo data.

Database

wilayas (reference, seeded)

Column	Type	Constraints / notes
code	char(2) PK	'01'..; must match the GeoJSON property used in System 5
name_ar, name_fr, name_en	varchar(80)	NOT NULL
aliases	text[]	alternative spellings used by `wilaya_normalize`
is_active	boolean	default true

Seed this table from the single wilaya list you choose (see open item 1). Do not guess names, codes or boundaries.

data_imports

Column	Type	Constraints / notes
id	uuid PK	
company_id	uuid FK	NOT NULL
import_type	import_type	social_posts | regional_metrics | sales_records
source_type	source_type	normally 'csv_import'
file_name	varchar(255)	
mode	varchar(10)	'append' or 'upsert' (policy chosen by the user, shown in the UI)
on_error	varchar(15)	'skip_invalid' or 'abort' (abort = import nothing if any row is invalid)
status	import_status	default 'previewed'
total_rows, accepted_rows, rejected_rows	int	
error_summary	jsonb	counts per error code
imported_by	uuid FK -> users	
created_at, completed_at	timestamptz	

data_import_errors

Column	Type	Constraints / notes
id, import_id FK (cascade), row_number int, column_name, error_code, message, raw_row jsonb		One row per rejected row/column; powers the downloadable error report

social_posts

Column	Type	Constraints / notes
id	uuid PK	
company_id	uuid FK	NOT NULL
platform	platform	NOT NULL
external_post_id	varchar(100)	nullable; UNIQUE (company_id, platform, external_post_id) WHERE NOT NULL
published_at	timestamptz	NOT NULL, not in the future
content_type	content_type	nullable
caption_public	text	nullable; public caption only; escape on display
likes, comments, shares, views, reach, impressions, clicks	int	ALL nullable; CHECK >= 0. Null means "not available", never 0
source_type	source_type	NOT NULL
import_id	uuid FK	nullable (null for manual entries)
created_at	timestamptz	index `(company_id, platform, published_at)`

regional_metrics

Column	Type	Constraints / notes
id	uuid PK	
company_id	uuid FK	NOT NULL
wilaya_code	char(2) FK -> wilayas	NOT NULL
period_start, period_end	date	NOT NULL, start <= end
metric_type	metric_type	impressions, clicks, conversions, spend, orders, revenue, followers, profit_margin
metric_value	numeric(18,4)	NOT NULL, >= 0
source_type, import_id		as above
unique		(company_id, wilaya_code, period_start, period_end, metric_type, source_type) for upsert

sales_records

Column	Type	Constraints / notes
id	uuid PK	
company_id	uuid FK	NOT NULL
order_date	date	NOT NULL
wilaya_code	char(2) FK	nullable
product_category	varchar(100)	nullable
order_count	int	NOT NULL, >= 0
revenue	numeric(14,2)	NOT NULL, >= 0 (DZD)
acquisition_cost	numeric(14,2)	nullable
source_type, import_id		as above

CSV templates (downloadable, one per type)

Type	Columns (header row, snake_case)	Required
social_posts	platform, external_post_id, published_at, content_type, caption_public, likes, comments, shares, views, reach, impressions, clicks	platform, published_at
regional_metrics	wilaya, period_start, period_end, metric_type, metric_value	all
sales_records	order_date, wilaya, product_category, order_count, revenue, acquisition_cost	order_date, order_count, revenue

Date format: `YYYY-MM-DD` (and `YYYY-MM-DD HH:mm` for `published_at`). `wilaya` accepts the code or any known name/alias; unknown values are rejected, never guessed.

Backend: endpoints

Method	Path	Access	Behavior / rules
GET	/api/imports/templates/:type	company_admin, analyst	Returns a CSV template with header row and one example row.
POST	/api/imports/:type/preview	company_admin, analyst	Multipart file + `mode` + `onError`. Validates the whole file, stores it in a temp folder (TTL 1 h), creates `data_imports` (status `previewed`). Returns first 20 rows, detected delimiter, column mapping, totals, accepted/rejected counts, first 50 errors.
POST	/api/imports/:id/commit	company_admin, analyst	Re-reads the temp file, inserts valid rows in batches of 500 inside a transaction (`append`: skip duplicates; `upsert`: ON CONFLICT update). Writes errors to `data_import_errors`. Final status `completed` / `completed_with_errors`. Deletes temp file.
GET	/api/imports	company roles	History (type, file, source, counts, status, date, user).
GET	/api/imports/:id	company roles	Detail with error summary.
GET	/api/imports/:id/errors.csv	company roles	Downloadable error report (row number, column, reason, raw values).
POST	/api/imports/:id/rollback	company_admin	Deletes rows created by that import (by `import_id`); marks import `cancelled`. Not allowed for upserted rows that overwrote older data unless noted in UI.
GET / POST	/api/posts	GET: company roles; POST: company_admin, analyst	Manual entry of PUBLIC post metrics (`source_type = public_manual`). List with filters platform/date.
PATCH / DELETE	/api/posts/:id	company_admin, analyst	Only rows of own company and only manual/imported rows.
POST / DELETE	/api/admin/demo-data/:companyId	super_admin	POST seeds synthetic rows (`source_type = synthetic_demo`) for a company; DELETE purges only synthetic rows.
GET	/api/wilayas	Authenticated	Reference list for selects and map labels.

Backend: service functions

Function	Behavior
csvValidate(buffer, schema) -> {validRows, errors, columns, delimiter}	= `csv_validate`. Handles BOM, `,` / `;` detection, header normalization (case/space), required columns, types, dates, negative values, in-file duplicates, max rows/size from settings. Unknown columns ignored with a warning.
wilayaNormalize(value) -> code | null	= `wilaya_normalize`. Trims, lowercases, strips diacritics, matches code, Arabic/French/English names and aliases. Returns null if not found (row is rejected).
importService.preview / commit / rollback / buildErrorCsv	Orchestrate validation, temp-file handling, transactions, logging (`import.created|completed|failed|rolled_back`).
templateService.build(type)	Generates the template CSV from the same schema object used for validation (single source of truth).
postService.create / update / remove	Manual public posts; forces `source_type = public_manual`.
demoService.seed(companyId) / purge(companyId)	Deterministic synthetic data (fixed random seed) across 12 months, all wilayas, 2 platforms; always `synthetic_demo`.

Frontend pages

Route	Page	Details
/data/imports	Import history	Table with status badges, counts, link to details; "New import" button.
/data/imports/new	Import wizard (stepper)	1 choose type + download template, 2 upload (drag and drop), 3 preview table with invalid cells highlighted and totals, 4 choose mode (append/upsert) and error policy, 5 confirm, 6 result (accepted/rejected, link to error CSV).
/data/imports/:id	Import details	Summary, error table, download errors CSV, rollback button (company admin).
/data/posts	Public posts	Table + add/edit modal for manual public post metrics; clear helper text that only public numbers belong here.
/admin/demo-data	Demo data (super admin)	Seed/purge per company with confirmation.

- Shared components: `FileDropzone`, `DataSourceBadge` (public / imported / official / DEMO), `DemoBanner`, `ImportErrorTable`.

Acceptance and tests

- Valid file: all rows accepted; counts match the file.
- Invalid file: rejected rows are listed with row number and reason; nothing fails silently.
- Re-importing the same posts does not create duplicates (`external_post_id` rule).
- Semicolon-delimited file with BOM and Arabic text imports correctly.
- A user cannot commit or read another company's import (404).
- Demo rows are flagged and excluded from `dataMode=real` queries.

System 5 - Geo Analytics

Goal: interactive map of Algerian wilayas colored by a selected metric, with details, comparison and ranking. Shows "No data" instead of guessing values.

Database

No new tables. Reads `regional_metrics`, `sales_records` and `wilayas`. Static asset: `web/public/geojson/algeria-wilayas.geojson` with a `code` property equal to `wilayas.code` (source and licence documented in the README).

Backend: endpoints

Method	Path	Access	Behavior / rules
GET	/api/analytics/geo	company roles	Query: `start`, `end`, `metric`, `dataMode`. Returns for each wilaya `{code, value|null, previous|null, changePct|null, rank|null}`, `missing[]`, `sources[]` (counts per source_type), `lastUpdated`, `warnings[]`.
GET	/api/analytics/geo/:wilayaCode	company roles	All metrics for the wilaya in the period, derived rates, trend by period.
GET	/api/analytics/geo/compare	company roles	`codes=16,31,25` (2 to 4). Side-by-side metrics and changes.

Supported metrics: direct (`impressions`, `clicks`, `conversions`, `spend`, `orders`, `revenue`, `followers`) and derived (`conversion_rate`, `cost_per_conversion`, `average_order_value`).

Backend: service functions

Function	Behavior
getGeoMetrics(companyId, start, end, metric, dataMode)	Aggregates by wilaya with SQL (`SUM`), restricted to the company and to the sources of the chosen `dataMode`.
rateConversionCalculate(conversions, clicks)	conversions / clicks x 100 via `divide_safe`; null when clicks is 0/missing.
conversionPerCostCalculate(spend, conversions)	spend / conversions; null if either value is missing or conversions is 0.
averageOrderValue(revenue, orders)	revenue / orders via `divide_safe`.
compareWilayaToPeriod(code, current, previous)	Growth % = (current - previous) / previous x 100; special state when previous is 0 (see System 6 `growthCalculate`).
wilayasRank(metrics, metricSelected)	Ranks only wilayas having a value; ties share rank; null values are listed under `missing`, not ranked last.

Derived metrics need numerator AND denominator from the same wilaya, same period and same source set; otherwise the result is null with reason. Public audience data does not represent sales; sales per wilaya come only from `sales_records` or imported metrics.

Frontend page: /analytics/geo

- Filter bar: date range (with "compare with previous period" toggle), metric select, `dataMode` toggle (Real / Demo), data-source badges.
- Leaflet choropleth: color scale with legend, grey hatch for no data, hover tooltip (name, value, change), click selects a wilaya.
- Side panel: selected wilaya details (all metrics, trend sparkline, derived rates with formulas), "Compare" button (select up to 4).
- Ranking table (sortable) + "missing data" list.
- Button **"Discover wilaya opportunities"** opens a modal that calls the System 6 opportunities endpoint and shows each wilaya with the reason for the recommendation.
- Empty state: "No regional data yet" with a button to the import wizard. Data-quality notice with sources and last update.
- Components: `ChoroplethMap`, `MetricSelect`, `DateRangePicker`, `WilayaDetailPanel`, `RankingTable`, `DataQualityNotice`.

Acceptance and tests

- Metric values match a hand calculation on a small known CSV.
- Zero clicks or zero conversions produce null, not Infinity/NaN.
- No data for a wilaya renders "No data" (not 0).
- Changing a company id in the URL/request cannot reveal another company's data.
- Demo and real data never appear together.

System 6 - Growth and Market Expansion

Goal: growth over time, key sales KPIs and an explainable ranking of wilaya expansion opportunities. Results describe the company's own data, not the whole Algerian market.

Database

No new tables. Weights and thresholds come from `system_settings` (`opportunity_weights`, `growth_min_periods`).

Formulas

```
Average order value (AOV) = total sales / number of orders          (null if orders = 0)
Sales growth %            = (current period sales - previous period sales) / previous period sales x 100
Conversion rate %         = conversions / clicks x 100
Cost per conversion       = spend / conversions
Special states for growth: previous = 0 and current > 0 -> state "new" (no percentage)
                           previous = 0 and current = 0 -> state "no_activity"
```

Opportunity score

Features per wilaya: `demand_growth` (orders growth %), `conversion_rate`, `leads_volume` (clicks or leads), `cac` (cost per acquisition, lower is better), `margin` (profit margin, optional). Default weights: 30 / 25 / 20 / 15 / 10 (editable). Steps: normalize each feature across wilayas to 0-100 (min-max; invert `cac`), multiply by weights, sum. Missing features are **never replaced by zero**: score is computed over available features with weights re-normalized and marked `partial` only if at least 3 of 5 features exist including `demand_growth` or `conversion_rate`; otherwise score = null and the reason lists the missing features. Low spend alone never makes a wilaya an opportunity.

Backend: endpoints

Method	Path	Access	Behavior / rules
GET	/api/analytics/growth/summary	company roles	Query: `start`, `end`, `granularity=day|week|month`, `metric=orders|revenue|followers`, `dataMode`. Returns series, previous-period series, totals, AOV, growth (value or special state), `periodsAvailable`, warnings if below `growth_min_periods`.
GET	/api/analytics/growth/opportunities	company roles	Returns ranked wilayas: `{code, score|null, status: ok|partial|insufficient, featuresUsed, featuresMissing, contributions, reasons[]}`. Optional `weights` override validated to sum 100.

Backend: service functions

Function	Behavior
growthCalculate(current, previous)	= `growth_calculate`; returns `{value, state}`.
periodByMetricsAggregate(records, period)	= `period_by_metrics_aggregate`; day/week/month buckets via SQL `date_trunc` in Africa/Algiers.
buildOpportunityFeatures(companyId, start, end)	Computes features per wilaya from `sales_records` and `regional_metrics` using `divide_safe`.
scoreNormalize / scoreOpportunityCalculate / scoreOpportunityExplain	Run in Flask (`POST /compute/opportunity-score`, Appendix C); Express validates the response with zod.
growthService.summary / opportunities	Orchestration, warnings and source metadata.

Frontend page: /analytics/growth

- Filters: date range, granularity, metric, `dataMode`.
- KPI cards: total sales, orders, AOV, sales growth (shows "New" / "No activity" labels for special states).
- Chart: current vs previous period (Chart.js line/bar).
- Opportunities table: rank, wilaya, score, status badge (OK / Partial / Insufficient), expandable "Why" row with contribution bars, factors raising/lowering the score and **missing factors listed explicitly**.
- Weights panel (read-only for analyst, editable preview with sum check); Reset to defaults.

- Disclaimer: "Based on your company data; not a measure of the whole Algerian market."
- Insufficient-data empty state with required data hints.

Acceptance and tests

- Growth and AOV match manual calculations; previous = 0 shows a special state.
- Wilayas with missing features are flagged, never scored as if the value were 0.
- Scores are reproducible and the explanation sums to the total.
- Fewer periods than the setting shows a warning instead of a trend claim.

System 7 - Scheduler Analytics and Internal Plan

Goal: find the best posting days/hours from the company's post data, with recommendations only when the sample is large enough, plus an internal plan of intended posts. No automatic publishing in v1.

Database

scheduled_items

Column	Type	Constraints / notes
id	uuid PK	
company_id	uuid FK	NOT NULL
platform	platform	NOT NULL
planned_at	timestamptz	NOT NULL
title	varchar(150)	NOT NULL
content_type	content_type	nullable
notes	text	nullable
status	varchar(12)	'planned' | 'done' | 'cancelled' (set manually; the system never posts)
created_by, created_at, updated_at		

Backend: endpoints

Method	Path	Access	Behavior / rules
GET	/api/analytics/scheduler	company roles	Query: `platform` (required), `start`, `end`, `contentType`, `dataMode`. Returns heatmap cells `{weekday, hour, postCount, value, median}`, ranked windows, `recommendations[]`, `warnings[]`, `metricUsed`, `sample {totalPosts, minRequired}`.
GET	/api/schedule/items	company roles	List with date range filter.
POST	/api/schedule/items	company_admin, analyst	Create a planned item.
PATCH / DELETE	/api/schedule/items/:id	company_admin, analyst	Edit or remove.

Backend: service functions

Function	Behavior
metricsPostPrepare(posts, tz)	= `metrics_post_prepare`; cleans dates, converts to Africa/Algiers, drops posts with no usable metric.
rateEngagementCalculate(interactions, impressions)	= `rate_engagement_calculate`; interactions / impressions x 100; null if impressions missing/0 (then this formula is NOT used).
Fallback metric `public_interactions`	likes + comments when reach/impressions are unavailable; the UI labels it "Public interactions (approximate)". Never mix the two metrics in one ranking.
hourWeekdayByPostsGroup(posts)	Groups by weekday (0-6) and hour (0-23).
windowsPostingRank(groups, metric, minSamples)	Ranks groups by median (tie-break by sample size); groups under `scheduler_min_posts_per_slot` are excluded from ranking but still shown greyed.
recommendationsSchedulerGenerate(stats, minSamples)	Top 3 windows with confidence label (low/medium/high by sample size) and warnings; returns no recommendation if total posts < `scheduler_min_total_posts`.
scheduleContentCreate(user, items)	= `schedule_content_create`; saves plan items only.

Computation runs in Flask (`POST /compute/scheduler`, Appendix C). Platforms are analyzed separately (Instagram first); numbers from different platforms are never combined.

Frontend pages

Route	Page	Details
/analytics/scheduler	Scheduler analysis	Platform tabs, content-type filter, date range, metric label (auto), 7x24 heatmap (custom grid; cell tooltip shows posts count and value; cells under minimum are hatched), best-window cards with confidence badge, warnings banner, "Add to plan" button.
/schedule	Internal plan	Week/list view of planned items; create/edit modal; statuses; note that nothing is published automatically. Week order option starting Saturday (Algerian week).

Acceptance and tests

- Weekday/hour grouping is correct around midnight and the timezone shift (UTC vs Africa/Algiers).
- Below threshold: warning and no recommendation (no empty claims).
- Switching platform changes data; platforms never merge.
- Engagement rate formula is not used when impressions are missing.

System 8 - Forecast (S-Curve Simulation)

Goal: simulate how a campaign could grow over time using a logistic curve, save several scenarios and compare them. The output is an assumption-based simulation, not a guarantee.

Database

campaign_simulations

Column	Type	Constraints / notes
id	uuid PK	
company_id	uuid FK	NOT NULL
name	varchar(120)	NOT NULL
budget	numeric(14,2)	nullable (DZD)
start_date	date	NOT NULL
duration_days	int	CHECK 1..365
assumptions_json	jsonb	`{targetAudience, K, r, t0, maxReachPct?}`
scenario_json	jsonb	computed series + milestones + summary + warnings
is_archived	boolean	default false (archived items do not count as active)
created_by, created_at, updated_at		

Model

```
y(t) = K / (1 + exp(-r * (t - t0)))
K  = upper limit (e.g. reachable audience x max reach %), must be > 0 and <= target audience
r  = growth rate per day (0 < r <= 5)
t0 = midpoint day (day when y = K/2)
Milestone day for fraction p of K:  t_p = t0 + ln(p / (1 - p)) / r      (p = 0.1, 0.5, 0.9)
Cost per result (optional) = budget / y(duration)       (null if y(duration) = 0 or no budget)
Warning if y(duration) < 0.9 K: "campaign ends before saturation"
```

Backend: endpoints

Method	Path	Access	Behavior / rules
POST	/api/forecast/simulate	company_admin, analyst	Validates assumptions, calls Flask, returns the result WITHOUT saving (live preview).
POST	/api/forecast/simulations	company_admin, analyst	Save a scenario (recomputes server-side; never trusts client-computed series).
GET	/api/forecast/simulations	company roles	List (non-archived by default).
GET / PATCH	/api/forecast/simulations/:id	GET: company roles; PATCH: analyst+	Read or update name/assumptions (recomputed).
DELETE	/api/forecast/simulations/:id	company_admin, analyst	Archive (soft delete).
POST	/api/forecast/compare	company roles	Body `{ids:[2..4]}`; returns aligned series and a summary table.
POST	/api/forecast/calibrate (optional)	analyst+	Fit K, r, t0 from observed cumulative points using Flask `curve_fit`; returns parameters and fit quality. Optional enhancement.

Backend: service functions

Function	Behavior
validateAssumptions(input)	zod schema + cross-field rules (K <= audience, duration, dates).
forecastService.simulate(assumptions)	Calls Flask `/compute/forecast`; validates the response shape.
simulationService.save / update / archive / list / get	Tenant-scoped CRUD; logs `simulation.created`.
simulationService.compare(ids)	Loads scenarios of the company only; aligns by day index.
statsService update	Admin dashboard counts non-archived simulations created in the last 30 days.

Frontend pages

Route	Page	Details
/forecast	Scenarios	Cards with name, date, key numbers; New scenario button; archive.
/forecast/new and /forecast/:id	Simulator	Left: assumptions form (audience, K or max reach %, r, t0, duration, start date, budget) with inline validation. Right: live S-curve chart (cumulative + daily increment), milestone cards (10/50/90%), warnings. Assumptions are always displayed next to the chart. Banner: "Estimate based on assumptions, not a guarantee". Save / Duplicate as new scenario.
/forecast/compare	Compare	Select 2-4 scenarios, overlay chart, summary table (final reach, days to 50%/90%, cost per result).

Acceptance and tests

- Values at t = t0 equal K/2; milestone days match the formula (unit test in Flask and in Express contract test).
- Invalid inputs (K <= 0, r <= 0, duration 0) are rejected with clear messages.
- Saved scenarios persist and reload identically.
- A scenario of another company cannot be loaded or compared (404).

System 9 - Reports and Hub

Goal: reproducible reports and the Hub landing page that links everything.

Database

reports

Column	Type	Constraints / notes
id	uuid PK	
company_id	uuid FK	NOT NULL
created_by	uuid FK -> users	
report_type	report_type	geo | growth | scheduler | forecast | combined
title	varchar(150)	
period_start, period_end	date	nullable for forecast-only reports
filters_json	jsonb	filters used
data_mode	varchar(5)	'real' or 'demo'
snapshot_json	jsonb	results frozen at generation time (makes reports reproducible)
sources_json	jsonb	which source types and counts fed each section
file_url	text	nullable (reserved for future PDF in Cloudinary raw storage)
created_at	timestamptz	generation date shown in the report

Backend: endpoints

Method	Path	Access	Behavior / rules
GET	/api/hub/summary	company roles	KPI summary, per-module data availability (has data / needs import), last imports, recent reports, data-quality notices.
POST	/api/reports	company_admin, analyst	Body `{type, title, start, end, filters, dataMode}`. Calls the module services, stores snapshot + sources, logs `report.created`.
GET	/api/reports	company roles	List with filters.
GET	/api/reports/:id	company roles	Full report (snapshot).
GET	/api/reports/:id/export.csv	company roles	CSV (UTF-8 BOM) for tabular sections; header lines show company, period, filters, sources, generated date.
DELETE	/api/reports/:id	company_admin	Delete.
GET	/api/reports/:id/export.pdf (later)	company roles	Optional: server-side PDF (e.g. Puppeteer) after v1; browser print stylesheet meanwhile.

Backend: service functions

Function	Behavior
reportService.generate(user, params)	Orchestrates geo/growth/scheduler/forecast services for the chosen period; refuses unsupported combinations; marks demo reports with a DEMO flag/watermark; never includes demo rows in `real` reports.
reportService.buildCsv(report)	Flattens each section; Arabic-safe encoding.
hubService.summary(companyId)	Counts and availability flags; cheap queries only.

Frontend pages

Route	Page	Details
/hub	SOMS Hub	Landing after login for company users: KPI cards, module tiles (Geo, Growth, Scheduler, Forecast, Reports) with data status, latest imports, recent reports, notices such as "no data imported yet".
/reports	Reports list	Filters by type/date; create button.
/reports/new	Report wizard	Choose module(s), period, filters, data mode; preview; generate.
/reports/:id	Report viewer	Title, generated date, filters, per-section charts and tables, data sources per section, approximation notes, Export CSV, Print (print stylesheet).

Acceptance and tests

- Report shows source of each dataset, generation date and filters.
- CSV opens correctly in Excel with Arabic text.
- Demo data appears only in reports labelled DEMO.
- Report of another company returns 404.
- Report content is unchanged after new data is imported (snapshot).

System 10 - Hardening, Testing and Delivery

Security checklist

- Helmet, CORS allowlist, rate limits, body/file limits verified with tests.
- All routes validated with zod; unknown fields stripped.
- Every company-data route has a tenant-isolation test (A cannot read/modify B).
- Cookies flags verified in production mode; refresh path scoped.
- No secrets in repo; `npm audit` clean or documented; Dependabot enabled.
- Uploads: mime + magic-byte check, size limits; CSV formula injection guard when exporting (prefix cells starting with = + - @ with an apostrophe).
- Logs contain no passwords/tokens; error responses do not leak stack traces in production.
- Flask not reachable from outside the Docker network.

Test strategy

Level	What	Tools
Unit	divide_safe, growth, AOV, conversion, opportunity score, S-curve, scheduler ranking, wilaya_normalize, csv_validate	Vitest/Jest, pytest
Integration	Auth flows, RBAC matrix, tenant isolation, import commit/rollback, report generation	Supertest + test database (separate Postgres DB)
Contract	Express <-> Flask request/response shapes	zod schemas + pytest
Frontend	Forms, guards, import wizard, i18n/RTL smoke	Vitest + RTL + MSW
E2E (optional)	Register -> approve -> login -> import -> see Geo -> create report	Playwright

Delivery

- README: prerequisites, `docker compose up`, env setup, migrate/seed commands, running the three services, running tests, known limitations.
- `.env.example` for all three apps; DB schema diagram exported from Prisma; OpenAPI/Swagger optional.
- Dockerfiles: api (node alpine, non-root user), analytics (gunicorn), web (build + Nginx/Caddy); `docker-compose.prod.yml` with internal network, volumes, restart policy.
- Backup script (`pg_dump` cron) and restore instructions.
- Limitations list: no Meta private data, no auto-publishing, no PDF (if postponed), Sentiments excluded.

Final acceptance checklist (from the source document)

- Every main button/form performs a clear function with understandable success and error messages.
- Company data persists after refresh and re-login.
- A company user cannot access another company's data by changing an id in the URL or request.
- Import shows accepted and rejected rows and never fails silently.
- Calculations match a manual calculation on a test sample.
- Missing data shows a proper message, not invented numbers or misleading charts.
- All demo data is clearly labelled and never mixed into real reports.
- UI works on suitable screen sizes with Arabic RTL support.
- Source code, run instructions, dependencies file and DB schema are delivered.

Appendix A - Flask Analytics Service Contract

Flask is a stateless compute service. It never touches the database and never receives user credentials. All calls come from Express with header `X-Internal-Key`. Adapt your existing algorithm code behind these endpoints (keep your function names in `algorithms/`). Every response must be pure JSON with explicit `null` for unavailable values.

Endpoint	Request (summary)	Response (summary)
GET /health	-	`{status: "ok"}` (no key needed)
POST /compute/forecast	`{targetAudience, K, r, t0, durationDays, startDate, budget?}`	`{series:[{day,date,cumulative,daily}], milestones:{p10,p50,p90}, summary:{finalValue,pctOfK,costPerResult|null}, warnings:[]}`
POST /compute/forecast/fit (optional)	`{observed:[{day,value}], K?}`	`{K, r, t0, rmse, warnings:[]}` using scipy curve_fit
POST /compute/scheduler	`{posts:[{publishedAt, likes, comments, shares, views, reach, impressions}], metric: "engagement_rate"|"public_interactions", minSamplesPerSlot, minTotalPosts, timezone}`	`{metricUsed, cells:[{weekday,hour,postCount,value,median}], ranking:[...], recommendations:[{weekday,hour,confidence,reason}], warnings:[]}`
POST /compute/opportunity-score	`{items:[{wilayaCode, features:{demandGrowth, conversionRate, leadsVolume, cac, margin}}], weights:{...}}` (null = missing)	`{results:[{wilayaCode, score|null, status, featuresUsed, featuresMissing, contributions, reasons:[]}]}`

- Validation with pydantic: reject NaN/Infinity, negative counts, absurd sizes (max 100,000 rows).
- Errors: HTTP 400 with `{error, details}`; Express maps them to VALIDATION_ERROR or INTERNAL_ERROR.
- Express timeout 10 s; Flask must be deterministic (same input, same output).

Appendix B - Antigravity Prompts

Use this workflow. Put this PDF (or the Markdown version) in `docs/` of the repository so the agent can read it, save the Master Prompt as the workspace rule/instructions file, then run one **System Prompt** at a time. Do not start the next system until the acceptance checklist of the current one passes.

B.1 Master prompt (paste once / save as workspace rules)

You are the lead full-stack engineer for SOMS (Social Marketing Services), a graduation project for the Algerian market. The full specification is in docs/SOMS_Build_Spec.md (same content as the PDF). Read it completely before writing code. It is the source of truth.

STACK (fixed): Express.js (JavaScript ESM) + JWT, PostgreSQL 16 in Docker with Prisma migrations, React (Vite) + Tailwind CSS, Cloudinary for images, Flask analytics microservice (Python), helmet, cors, zod, argon2, express-rate-limit, multer, csv-parse, pino, react-i18next (AR/FR/EN with RTL), TanStack Query, React Hook Form, Chart.js, Leaflet.

NON-NEGOTIABLE RULES:
1. Build ONE system at a time, exactly as described in the spec section I name. Do not start other systems or add unrelated features.
2. Tenant isolation is enforced on the server: company id comes only from the verified JWT. Resource lookups use id AND company_id; cross-company access returns 404.
3. Never invent data or numbers. Missing data or zero denominators return null with a reason; the UI shows "No data / Insufficient data".
4. Every data row has source_type. Demo data (synthetic_demo) is never mixed with real data; dataMode=real|demo.
5. Never store secrets in code. Use env vars. Hash passwords with argon2id. Never log passwords or tokens.
6. Use the standard response envelope and error codes from the spec. Validate every request with zod.
7. Flask is internal only (X-Internal-Key), stateless, no database access.
8. Do not use or build anything for the Sentiments module. No Meta/Instagram private data access or scraping.
9. Keep the UI RTL-ready (Tailwind logical utilities) and translated through i18n files (no hard-coded user-facing strings).
10. Write tests for the system you build (unit + integration, including tenant isolation where relevant). Run them and fix failures before finishing.

WORKFLOW FOR EVERY TASK:
A. Restate the system scope and list the files you will create or change (short plan). Ask me only if something in the spec is ambiguous or an open item blocks you.
B. Implement database migration -> backend -> frontend -> Flask (only if the system needs it).
C. Run the tests and the app; show the commands and results.
D. Finish with: what was built, how to run/test it, a manual test checklist for me, known limitations, and anything that deviates from the spec (with the reason).
If you want to deviate from the spec, stop and explain first.

B.2 System prompt template (use for Systems 1 to 10)

Implement SYSTEM [N] - [NAME] from docs/SOMS_Build_Spec.md, section "System [N]". Follow the Master rules.

Scope: only what the spec lists for this system (database tables, backend endpoints and service functions, frontend pages and components, Flask endpoints if listed).
Already completed systems (do not rewrite unless needed to integrate): [LIST, e.g. 0, 1, 2].
Open decisions I have made: [e.g. wilaya list = 58 official wilayas; GeoJSON file at web/public/geojson/algeria-wilayas.geojson; platforms = instagram + facebook].

Steps:
1. Write a short plan (files, migration name, endpoints, pages).
2. Create the Prisma migration exactly matching the spec tables, constraints and indexes.
3. Implement backend modules (routes -> controller -> service -> zod schema) with permissions from the matrix and tenant scope middleware.
4. Implement the frontend pages and components with i18n keys (ar/fr/en), loading/error/empty states, and role-based navigation.
5. Add tests: unit tests for calculations/helpers, integration tests (Supertest) for each endpoint including 401/403/404 and cross-company access, frontend tests for the main forms.
6. Run everything. Fix failures.
7. Report: summary, commands to run, manual test checklist (step-by-step, with expected results), deviations, and the acceptance checklist from the spec marked pass/fail.

Do not continue to the next system.

B.3 Ready-to-use prompt: System 0 and System 1

TASK 1 - SYSTEM 0 (Foundation). Read docs/SOMS_Build_Spec.md sections 1-5 and "System 0". Create the repository structure (api/, analytics/, web/), docker-compose.yml with postgres:16 (volume, healthcheck) and optional pgadmin profile, .env.example files, Express skeleton with the middleware order, env validation, error handling, health endpoint, Prisma init, Flask skeleton with internal key check, and the React shell with Tailwind, router, layouts, i18n (AR default, RTL switching), axios client and the shared UI kit. Provide README commands to start everything. Stop after System 0 and report using the workflow in the Master rules.

TASK 2 - SYSTEM 1 (Authentication and Authorization). Read "System 1". Implement companies, users, refresh_tokens migrations; seed ensureSuperAdmin; login/refresh/logout/me/change-password with JWT access token (15 min, in memory on the client) and rotating hashed refresh token in an httpOnly cookie with reuse detection; account lockout after 5 failed attempts; requireAuth, requireRole, resolveCompanyScope middleware; permissions matrix module; frontend login page, AuthProvider with silent refresh, axios 401 interceptor, ProtectedRoute/RoleGuard, profile page. Add the reusable test helper for cross-company access (expects 404). Run tests and report with the acceptance checklist.

B.4 Review prompt (run after each system)

Review the code you just wrote for SYSTEM [N] against docs/SOMS_Build_Spec.md. Produce a table: requirement | implemented (yes/no/partial) | file/evidence. Then audit specifically for: (1) any endpoint that reads or writes company data without company_id scoping, (2) places where null/zero-denominator cases could output 0, NaN or Infinity, (3) secrets or tokens in code or logs, (4) missing zod validation, (5) hard-coded UI strings or LTR-only styles, (6) missing tests. Fix every issue you find, rerun the tests, and list what changed.